MODULE 7: PROJECT PART 2

GitHub Repository Link: https://github.com/devcnx/csd430

styles.css

```
/*
    styles.css - External CSS for Module 4 assignment
    Author: Brittaney Perry-Morgan
    Description: Styles for the JavaBean movie data display.
 */

/* ==========================
   1. GLOBAL STYLES
   ========================== */

*,
html,
body {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: linear-gradient(120deg, #f8fafc 0%, #e0e7ef 100%);
    margin: 0;
    padding: 20px;
    min-height: 100vh;
}

/* ==========================
   2. LAYOUT & CONTAINER
   ========================== */

.container,
.top-title {
    max-width: 900px;
    margin: 0 auto;
    padding: 20px;
}
```

```
.container {
    background: #fff;
    border-radius: 12px;
    box-shadow: 0 4px 24px rgba(0, 0, 0, 0.08);
    margin-bottom: 20px;
}

.top-title {
    text-align: center;
    background: linear-gradient(135deg, #2a3a5e 0%, #1a2540 100%);
    color: #fff;
}

.top-title h1 {
    color: #fff;
    margin: 0 0 10px 0;
}

.top-title h2 {
    color: #e0e7ef;
    margin: 0;
    font-weight: 400;
}

.top-title h2 em {
    font-style: normal;
    color: #a3d9ff;
}

/* ========================
   3. TYPOGRAPHY
   ======================== */

h1,
h2,
h3 {
    color: #2a3a5e;
    margin: 20px 0 15px 0;
    text-transform: uppercase;
}
```

```
h1 {
    font-weight: 200;
    font-size: 1.8rem;
}

h2 {
    font-weight: 400;
    font-size: 1.4rem;
}

h3 {
    font-weight: 600;
    font-size: 1.1rem;
    color: #1a2540;
    border-bottom: 2px solid #e3eaf6;
    padding-bottom: 8px;
}

p.description {
    color: #ffffff;
    line-height: 1.7;
    margin: 15px auto;
    padding: 0 20px;
}

/* ========================
   4. LISTS
   ======================== */

.field-descriptions {
    list-style: none;
    padding: 0 20px;
}

.field-descriptions li {
    padding: 10px 15px;
    margin: 8px 0;
    background: #f1f5fa;
    border-radius: 6px;
    border-left: 4px solid #2a3a5e;
    color: #444;
```

```
}

/* ========================
   5. FORMS
   ========================= */

.form-table {
    width: 100%;
    border-collapse: collapse;
    margin: 20px;
    background: #fff;
}

.form-table tr {
    background: #fff;
}

.form-table td {
    border: 1px solid #e3eaf6;
    padding: 12px 15px;
    vertical-align: top;
}

.form-table td:first-child {
    background: #f9fbfd;
    width: 25%;
    font-weight: 600;
    color: #1a2540;
    vertical-align: middle;
}

.form-table td:last-child {
    width: 75%;
}

.form-table label {
    display: block;
    margin-bottom: 5px;
    font-size: 0.9rem;
    color: #333;
}
```

```
.form-table input[type="text"],
.form-table input[type="email"],
.form-table input[type="tel"],
.form-table input[type="number"],
.form-table input[type="date"],
.form-table select,
.form-table textarea {
    width: 100%;
    padding: 10px 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 1rem;
    font-family: 'Segoe UI', Arial, sans-serif;
    box-sizing: border-box;
}

.form-table select {
    background: #fff;
    cursor: pointer;
}

.form-table textarea {
    resize: vertical;
    min-height: 80px;
}

.form-table input[type="checkbox"] {
    width: auto;
    margin-right: 8px;
}

/* ========================
   6. NAVIGATION
   ======================== */

nav ul {
    list-style: none;
    padding: 0 20px;
}
```

```
nav ul li {
    margin: 10px 0;
}

nav ul li a {
    color: #2a3a5e;
    text-decoration: none;
    font-weight: 600;
    padding: 8px 12px;
    background: #f1f5fa;
    border-radius: 6px;
    transition: all 0.2s ease;
}

nav ul li a:hover {
    background: #2a3a5e;
    color: #fff;
}

.nav-desc {
    display: block;
    color: #666;
    font-size: 0.85rem;
    margin-top: 8px;
    font-weight: 400;
}

/* ========================
   6. FORM ACTIONS (for button links)
   ========================= */

.form-actions {
    margin: 30px 20px;
    padding: 15px;
    background: #f9fbfd;
    border-radius: 8px;
    text-align: center;
}

.btn-submit,
.btn-reset {
```

```
    padding: 12px 24px;
    margin: 0 10px;
    font-size: 1rem;
    font-weight: 600;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    transition: all 0.2s ease;
    text-decoration: none;
    display: inline-block;
}

.btn-submit {
    background: #2a3a5e;
    color: #fff;
}

.btn-submit:hover {
    background: #1a2540;
    transform: translateY(-1px);
    box-shadow: 0 4px 12px rgba(42, 58, 94, 0.3);
}

.btn-reset {
    background: #e3eaf6;
    color: #2a3a5e;
}

.btn-reset:hover {
    background: #cdd8eb;
}

/* ========================
   7. FORM ACTIONS (for button links)
   ========================= */

.form-actions {
    margin: 30px 20px;
    padding: 15px;
    background: #f9fbfd;
    border-radius: 8px;
```

```
    text-align: center;
}

.btn-submit,
.btn-reset {
    padding: 12px 24px;
    margin: 0 10px;
    font-size: 1rem;
    font-weight: 600;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    transition: all 0.2s ease;
    text-decoration: none;
    display: inline-block;
}

.btn-submit {
    background: #2a3a5e;
    color: #fff;
}

.btn-submit:hover {
    background: #1a2540;
    transform: translateY(-1px);
    box-shadow: 0 4px 12px rgba(42, 58, 94, 0.3);
}

.btn-reset {
    background: #e3eaf6;
    color: #2a3a5e;
}

.btn-reset:hover {
    background: #cdd8eb;
}

/* ==========================
   8. TABLES
   ========================== */
```

```
table {
    width: 100%;
    border-collapse: collapse;
    margin: 20px;
    background: #fff;
}

thead {
    background: #2a3a5e;
    color: #fff;
}

th,
td {
    border: 1px solid #e3eaf6;
    padding: 12px 15px;
    text-align: left;
}

th {
    font-weight: 600;
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

td {
    font-size: 0.95rem;
    color: #333;
    vertical-align: top;
}

tr:nth-child(even) {
    background: #f9fbfd;
}

tr:hover {
    background: #e8f0ff;
    transition: background 0.2s ease;
}
```

```
/* ========================
   9. FOOTER
   ======================== */

footer {
    text-align: center;
    padding: 20px;
    color: #666;
    font-size: 0.9rem;
}

footer p {
    margin: 0;
}

/* ========================
   10. RESPONSIVE DESIGN
   ======================== */

@media (max-width: 768px) {
    body {
        padding: 10px;
    }

    .container,
    .top-title {
        padding: 15px;
    }

    h1 {
        font-size: 1.4rem;
    }

    h2 {
        font-size: 1.1rem;
    }

    h3 {
        font-size: 1rem;
    }
```

```
.form-table {
    margin: 10px 0;
    font-size: 0.85rem;
}

.form-table td {
    padding: 8px 10px;
}

.form-table td:first-child {
    font-size: 0.85rem;
}

.form-table input[type="text"],
.form-table input[type="email"],
.form-table input[type="tel"],
.form-table input[type="number"],
.form-table input[type="date"],
.form-table select,
.form-table textarea {
    font-size: 0.9rem;
    padding: 8px;
}

table {
    margin: 10px 0;
    font-size: 0.85rem;
}

th,
td {
    padding: 8px 10px;
}

p.description {
    padding: 0 10px;
}

.field-descriptions {
    padding: 0 10px;
}
```

```
    .field-descriptions li {
        padding: 8px 12px;
    }

    nav ul {
        padding: 0 10px;
    }

    .form-actions {
        margin: 20px 10px;
        padding: 10px;
    }

    .btn-submit,
    .btn-reset {
        padding: 10px 16px;
        font-size: 0.9rem;
        margin: 5px;
    }
}
```

index.jsp

```
<%--
    CSD430 Server-Side Development

    Module 7 Project Part 2: CRUD-CREATE Operations
    Author: Brittaney Perry-Morgan

    Description: Course index page with navigation to movie database
    application demonstrating JDBC connectivity and JavaBeans for
    CRUD-CREATE operations.

    index.jsp - Course Index Page
--%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Module 7 Project Part 2 - CRUD-CREATE Operations</title>
```

```
    <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body>
    <div class="container top-title">
        <h1>Module 7 Project Part 2</h1>
        <h2>JDBC & JavaBeans - <em>CRUD-CREATE Operations</em></h2>
        <p class="description">Author: Brittaney Perry-Morgan</p>
    </div>

    <div class="container">
        <h3>Movie Database CRUD Operations</h3>
        <p class="description">
            This web application demonstrates database connectivity using JDBC
            and JavaBeans for Create, Read, Update, and Delete (CRUD) operations.
            Module 7 focuses on <strong>CREATE</strong> operations - adding new
            records to the database.
        </p>

        <h3>Database Information</h3>
        <ul class="field-descriptions">
            <li><strong>Database Name:</strong> CSD430</li>
            <li><strong>Table Name:</strong> brittaney_movies_data</li>
            <li><strong>Username:</strong> student1</li>
            <li><strong>Topic:</strong> Movies</li>
            <li><strong>Fields:</strong> 6 (movie_id, title, genre, release_year, rating, director)</li>
        </ul>

        <h3>Navigation - CREATE Operations</h3>
        <nav>
            <ul>
                <li>
                    <a href="movieForm.jsp">Add New Movie</a>
                    <span class="nav-desc">Form to add a new movie record to the database</span>
                </li>
                <li>
                    <a href="movieList.jsp">View All Movies</a>
                    <span class="nav-desc">Display all movie records in table format</span>
                </li>
            </ul>
        </nav>
    </div>
```

```
    <footer>
        <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
    </footer>
</body>
</html>
```

MovieBean.java

```
package csd430;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

/**
 * MovieBean - JDBC JavaBean for CRUD Operations.
 *
 * <p>
 * This JavaBean provides database connectivity using JDBC to perform
 * CRUD (Create, Read, Update, Delete) operations on movie records
 * in a MySQL database. This bean is used across multiple modules:
 * </p>
 * <ul>
 *   <li>Module 5&amp;amp;6 Project Part 1: CRUD-READ operations</li>
 *   <li>Module 7 Project Part 2: CRUD-CREATE operations</li>
 * </ul>
 *
 * <p>
 * The bean follows JavaBean conventions with getter methods for each property
 * and uses PreparedStatement to prevent SQL injection attacks.
 * </p>
 *
 * <p>
 * Database configuration is loaded from environment variables or a .env file
```

```
 * with fallback defaults for local development.
 * </p>
 *
 * @author Brittaney Perry-Morgan
 * @version 2.0
 * @since CSD430 Module 5 &amp; 6 Project Part 1
 */
public class MovieBean {

    // ========================
    // Database Configuration
    // ========================

    /** Database connection URL (e.g., jdbc:mysql://localhost:3306/CSD430) */
    private static String DB_URL;

    /** Database username for authentication */
    private static String DB_USER;

    /** Database password for authentication */
    private static String DB_PASSWORD;

    /**
     * Static initializer block that loads database configuration.
     *
     * <p>
     * Configuration is loaded in the following order of precedence:
     * </p>
     * <ol>
     *   <li>Environment variables (DB_URL, DB_USER, DB_PASSWORD)</li>
     *   <li>.env file in the application root directory</li>
     *   <li>Default fallback values</li>
     * </ol>
     */
    static {
        loadEnvironmentVariables();
    }

    /**
     * Loads database configuration from environment variables or .env file.
     *
```

```
 * <p>
 * This method attempts to read configuration from system environment
 * variables first. If not found, it looks for a .env file in the
 * application directory. If neither source provides configuration,
 * default values are used.
 * </p>
 */
private static void loadEnvironmentVariables() {
    // Try environment variables first
    DB_URL = System.getenv("DB_URL");
    DB_USER = System.getenv("DB_USER");
    DB_PASSWORD = System.getenv("DB_PASSWORD");

    // Fall back to .env file if environment variables not set
    if (DB_URL == null || DB_USER == null || DB_PASSWORD == null) {
        String envFile = System.getProperty("user.dir") + "/.env";
        File file = new File(envFile);

        if (file.exists()) {
            try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    // Skip empty lines and comments
                    if (line.trim().isEmpty() || line.trim().startsWith("#")) {
                        continue;
                    }

                    String[] parts = line.split("=", 2);
                    if (parts.length == 2) {
                        String key = parts[0].trim();
                        String value = parts[1].trim();

                        switch (key) {
                            case "DB_URL":
                                DB_URL = value;
                                break;
                            case "DB_USER":
                                DB_USER = value;
                                break;
                            case "DB_PASSWORD":
                                DB_PASSWORD = value;
```

```
                        break;
                    default:
                        // Ignore unknown keys
                        break;
                }
            }
        }
    } catch (IOException e) {
        System.err.println("Error loading .env file: " + e.getMessage());
    }
  }
}

  // Apply default values if still null
  if (DB_URL == null) {
    DB_URL = "jdbc:mysql://localhost:3306/CSD430";
  }
  if (DB_USER == null) {
    DB_USER = "student1";
  }
  if (DB_PASSWORD == null) {
    DB_PASSWORD = "pass";
  }
}

// ========================
// Instance Fields
// ========================

/** Active database connection for this bean instance */
private Connection connection;

/** List of movie IDs retrieved from the database */
private List<Integer> movieIds;

/** ResultSet containing the current movie records */
private ResultSet resultSet;

/** Error message from the last failed operation */
private String errorMessage;
```

```
// ========================
// Constructors
// ========================

/**
 * Constructs a new MovieBean instance.
 *
 * <p>
 * Initializes the movie IDs list and sets the error message to null.
 * Database connection is established lazily when needed.
 * </p>
 */
public MovieBean() {
    movieIds = new ArrayList<>();
    errorMessage = null;
}

// ========================
// Database Connection Methods
// ========================

/**
 * Establishes a connection to the MySQL database.
 *
 * <p>
 * This method loads the MySQL JDBC driver and creates a connection
 * using the configured URL, username, and password. It supports
 * both the newer com.mysql.cj.jdbc.Driver and legacy com.mysql.jdbc.Driver.
 * </p>
 *
 * @return {@code true} if connection was successful, {@code false} otherwise
 */
public boolean connect() {
    try {
        // Try newer driver first, fall back to legacy driver
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            Class.forName("com.mysql.jdbc.Driver");
        }
```

```
        connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
        return true;

    } catch (ClassNotFoundException e) {
        errorMessage = "MySQL Driver not found: " + e.getMessage();
        return false;
    } catch (SQLException e) {
        errorMessage = "Database connection error: " + e.getMessage();
        return false;
    }
}

/**
 * Closes the database connection and releases resources.
 *
 * <p>
 * This method should be called when the bean is no longer needed
 * to prevent resource leaks. It safely handles null connections
 * and result sets.
 * </p>
 */
public void disconnect() {
    try {
        if (resultSet != null) {
            resultSet.close();
            resultSet = null;
        }
        if (connection != null) {
            connection.close();
            connection = null;
        }
    } catch (SQLException e) {
        errorMessage = "Error closing connection: " + e.getMessage();
    }
}

// ==========================
// CRUD-READ Methods
// ==========================

/**
```

```
 * Retrieves all movie IDs from the database.
 *
 * <p>
 * This method queries the database for all movie_id values and
 * populates the internal movieIds list. The IDs are ordered
 * ascending for consistent dropdown display.
 * </p>
 *
 * <p>
 * If no connection exists, this method will attempt to connect
 * automatically.
 * </p>
 *
 * @return {@code true} if IDs were retrieved successfully,
 *         {@code false} if an error occurred
 */
public boolean getAllMovieIds() {
    // Ensure connection exists
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "SELECT movie_id FROM brittaney_movies_data ORDER BY movie_id";
        PreparedStatement stmt = connection.prepareStatement(query);
        resultSet = stmt.executeQuery();

        // Clear existing list and populate with new IDs
        movieIds.clear();
        while (resultSet.next()) {
            movieIds.add(resultSet.getInt("movie_id"));
        }

        // Clean up resources
        resultSet.close();
        stmt.close();
        return true;

    } catch (SQLException e) {
```

```
        errorMessage = "Error fetching movie IDs: " + e.getMessage();
        return false;
    }
}

/**
 * Retrieves a single movie record by its primary key.
 *
 * <p>
 * This method queries the database for a specific movie using
 * the movie_id parameter. If found, the ResultSet is stored
 * internally and can be accessed via the getter methods.
 * </p>
 *
 * <p>
 * Uses PreparedStatement to prevent SQL injection attacks.
 * </p>
 *
 * @param movieId the primary key of the movie to retrieve
 * @return {@code true} if the movie was found, {@code false} otherwise
 */
public boolean getMovieById(int movieId) {
    // Ensure connection exists
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "SELECT * FROM brittaney_movies_data WHERE movie_id = ?";
        PreparedStatement stmt = connection.prepareStatement(query);
        stmt.setInt(1, movieId);
        resultSet = stmt.executeQuery();

        if (!resultSet.next()) {
            errorMessage = "Movie not found with ID: " + movieId;
            resultSet.close();
            stmt.close();
            return false;
        }
```

```
        return true;

    } catch (SQLException e) {
        errorMessage = "Error fetching movie: " + e.getMessage();
        return false;
    }
}

/**
 * Retrieves all movie records from the database.
 *
 * <p>
 * This method queries the database for all movies and stores
 * the ResultSet internally. Use {@link #next()} to iterate
 * through records and getter methods to access field values.
 * </p>
 *
 * @return {@code true} if records were retrieved successfully,
 *         {@code false} if an error occurred
 */
public boolean getAllMovies() {
    // Ensure connection exists
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "SELECT * FROM brittaney_movies_data ORDER BY movie_id";
        PreparedStatement stmt = connection.prepareStatement(query);
        resultSet = stmt.executeQuery();
        return true;

    } catch (SQLException e) {
        errorMessage = "Error fetching movies: " + e.getMessage();
        return false;
    }
}
```

```java
// ========================
// CRUD-CREATE Methods
// ========================

/**
 * Inserts a new movie record into the database.
 *
 * <p>
 * This method creates a new movie record with the provided field values.
 * The movie_id is auto-generated by the database (AUTO_INCREMENT).
 * Uses PreparedStatement to prevent SQL injection attacks.
 * </p>
 *
 * @param title the title of the movie
 * @param genre the genre of the movie
 * @param releaseYear the year the movie was released
 * @param rating the MPAA rating (G, PG, PG-13, R, NC-17)
 * @param director the director of the movie
 * @return {@code true} if the record was inserted successfully,
 *         {@code false} if an error occurred
 */
public boolean insertMovie(String title, String genre, int releaseYear, String rating, String director) {
    // Ensure connection exists
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "INSERT INTO brittaney_movies_data (title, genre, release_year, rating, director) VALUES (?, ?, ?, ?, ?)";
        PreparedStatement stmt = connection.prepareStatement(query);
        stmt.setString(1, title);
        stmt.setString(2, genre);
        stmt.setInt(3, releaseYear);
        stmt.setString(4, rating);
        stmt.setString(5, director);

        int rowsAffected = stmt.executeUpdate();
        stmt.close();
        return rowsAffected > 0;
```

```
    } catch (SQLException e) {
        errorMessage = "Error inserting movie: " + e.getMessage();
        return false;
    }
}

// ========================
// Getter Methods (JavaBean Properties)
// ========================

/**
 * Returns the list of all movie IDs.
 *
 * <p>
 * This list is populated by the {@link #getAllMovieIds()} method.
 * </p>
 *
 * @return a List of Integer movie IDs, or an empty list if not yet populated
 */
public List<Integer> getMovieIds() {
    return movieIds;
}

/**
 * Returns the movie_id of the current record.
 *
 * <p>
 * This method should only be called after a successful call to
 * {@link #getMovieById(int)} or during iteration with {@link #next()}.
 * </p>
 *
 * @return the movie ID, or 0 if an error occurs
 */
public int getMovieId() {
    try {
        return resultSet.getInt("movie_id");
    } catch (SQLException e) {
        return 0;
    }
}
```

```
/**
 * Returns the title of the current movie record.
 *
 * @return the movie title, or an empty string if an error occurs
 */
public String getTitle() {
    try {
        return resultSet.getString("title");
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the genre of the current movie record.
 *
 * @return the movie genre, or an empty string if an error occurs
 */
public String getGenre() {
    try {
        return resultSet.getString("genre");
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the release year of the current movie record.
 *
 * @return the release year, or 0 if an error occurs
 */
public int getReleaseYear() {
    try {
        return resultSet.getInt("release_year");
    } catch (SQLException e) {
        return 0;
    }
}

/**
```

```
 * Returns the MPAA rating of the current movie record.
 *
 * @return the rating (e.g., "PG-13", "R"), or an empty string if an error occurs
 */
public String getRating() {
    try {
        return resultSet.getString("rating");
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the director of the current movie record.
 *
 * @return the director name, or an empty string if an error occurs
 */
public String getDirector() {
    try {
        return resultSet.getString("director");
    } catch (SQLException e) {
        return "";
    }
}

// ========================
// Navigation Methods
// ========================

/**
 * Advances to the next record in the ResultSet.
 *
 * <p>
 * This method is used to iterate through multiple records returned
 * by {@link #getAllMovies()}. Returns false when there are no more
 * records.
 * </p>
 *
 * @return {@code true} if there is a next record, {@code false} otherwise
 */
public boolean next() {
```

```
        try {
            return resultSet.next();
        } catch (SQLException e) {
            errorMessage = "Error navigating result set: " + e.getMessage();
            return false;
        }
    }

    // ========================
    // Error Handling
    // ========================

    /**
     * Returns the error message from the last failed operation.
     *
     * <p>
     * This method should be called after a method returns {@code false}
     * to determine the cause of the failure.
     * </p>
     *
     * @return the error message, or {@code null} if no error occurred
     */
    public String getErrorMessage() {
        return errorMessage;
    }
}
```

movieForm.jsp

```
<%--
    CSD430 Server-Side Development

    Module 7 Project Part 2: CRUD-CREATE Operations
    Author: Brittaney Perry-Morgan

    Description: Form page for adding new movie records to the database.
    Uses JDBC and JavaBeans for database connectivity.

    movieForm.jsp - Add Movie Form Page
--%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="csd430.MovieBean" %>
<!DOCTYPE html>
```

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Add Movie - CSD430</title>
    <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body>
    <div class="container top-title">
        <h1>Module 7 Project Part 2</h1>
        <h2>Movie Database - <em>CREATE Operation</em></h2>
        <p class="description">Add a new movie record to the database.</p>
    </div>

    <div class="container">
<%
        MovieBean movieBean = new MovieBean();
        String successMessage = null;
        String errorMessage = null;

        // Process form submission
        if (request.getMethod().equalsIgnoreCase("POST")) {
            String title = request.getParameter("title");
            String genre = request.getParameter("genre");
            String releaseYearStr = request.getParameter("release_year");
            String rating = request.getParameter("rating");
            String director = request.getParameter("director");

            // Validate all fields are present
            if (title != null && !title.isEmpty() &&
                genre != null && !genre.isEmpty() &&
                releaseYearStr != null && !releaseYearStr.isEmpty() &&
                rating != null && !rating.isEmpty() &&
                director != null && !director.isEmpty()) {

                try {
                    int releaseYear = Integer.parseInt(releaseYearStr);
                    boolean connected = movieBean.connect();

                    if (connected) {
                        boolean success = movieBean.insertMovie(title, genre, releaseYear, rating, director);
```

```
                movieBean.disconnect();

                if (success) {
                    successMessage = "Movie \"" + title + "\" has been added successfully!";
                } else {
                    errorMessage = "Error adding movie: " + movieBean.getErrorMessage();
                }
            } else {
                errorMessage = "Database connection error: " + movieBean.getErrorMessage();
            }
        } catch (NumberFormatException e) {
            errorMessage = "Invalid release year. Please enter a valid number.";
        }
    } else {
        errorMessage = "All fields are required. Please fill in all the details.";
    }
}

// Display success message
if (successMessage != null) {
%>
<div class="success-message">
    <p><%= successMessage %></p>
</div>
<%
}

// Display error message
if (errorMessage != null) {
%>
<div class="error-message">
    <p><%= errorMessage %></p>
</div>
<%
}
%>

<h3>Add New Movie</h3>
<p class="description">Fill in the details below to add a new movie to the database.</p>

<form action="movieForm.jsp" method="post">
```

```
        <table class="form-table">
            <tr>
                <td><label for="title">Movie Title:</label></td>
                <td><input type="text" id="title" name="title" required placeholder="Enter movie title"></td>
            </tr>
            <tr>
                <td><label for="genre">Genre:</label></td>
                <td>
                    <select id="genre" name="genre" required>
                        <option value="">-- Select Genre --</option>
                        <option value="Action">Action</option>
                        <option value="Comedy">Comedy</option>
                        <option value="Crime">Crime</option>
                        <option value="Drama">Drama</option>
                        <option value="Fantasy">Fantasy</option>
                        <option value="Horror">Horror</option>
                        <option value="Romance">Romance</option>
                        <option value="Sci-Fi">Sci-Fi</option>
                        <option value="Thriller">Thriller</option>
                    </select>
                </td>
            </tr>
            <tr>
                <td><label for="release_year">Release Year:</label></td>
                <td><input type="number" id="release_year" name="release_year" min="1888" max="2030" required placeholder="e.g.,
2024"></td>
            </tr>
            <tr>
                <td><label for="rating">MPAA Rating:</label></td>
                <td>
                    <select id="rating" name="rating" required>
                        <option value="">-- Select Rating --</option>
                        <option value="G">G</option>
                        <option value="PG">PG</option>
                        <option value="PG-13">PG-13</option>
                        <option value="R">R</option>
                        <option value="NC-17">NC-17</option>
                    </select>
                </td>
            </tr>
            <tr>
```

```
                <td><label for="director">Director:</label></td>
                <td><input type="text" id="director" name="director" required placeholder="Enter director name"></td>
            </tr>
        </table>

        <div class="form-actions">
            <button type="submit" class="btn-submit">Add Movie</button>
            <button type="reset" class="btn-reset">Clear Form</button>
        </div>
    </form>

    <h3>Field Descriptions</h3>
    <ul class="field-descriptions">
        <li><strong>Movie Title:</strong> The name of the movie</li>
        <li><strong>Genre:</strong> The category or type of movie (Action, Comedy, etc.)</li>
        <li><strong>Release Year:</strong> The year the movie was released</li>
        <li><strong>MPAA Rating:</strong> Motion picture rating (G, PG, PG-13, R, NC-17)</li>
        <li><strong>Director:</strong> The director of the movie</li>
    </ul>

    <div class="form-actions">
        <a href="movieList.jsp" class="btn-submit">View All Movies</a>
        <a href="index.jsp" class="btn-reset">Back to Course Index</a>
    </div>
  </div>

  <footer>
    <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
  </footer>
</body>
</html>
```

movieList.jsp

```
<%--
    CSD430 Server-Side Development

    Module 7 Project Part 2: CRUD-CREATE Operations
    Author: Brittaney Perry-Morgan

    Description: Displays all movies from the database in an HTML table.
    Demonstrates READ operation after CREATE to verify the new record.
```

```
    movieList.jsp - Movie List Display Page
--%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="csd430.MovieBean" %>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Movie List - CSD430</title>
    <link rel="stylesheet" type="text/css" href="css/styles.css">
    <style>
        .success-message {
            background-color: #d4edda;
            border: 1px solid #c3e6cb;
            color: #155724;
            padding: 15px;
            margin: 20px;
            border-radius: 6px;
            text-align: center;
        }
        .error-message {
            background-color: #f8d7da;
            border: 1px solid #f5c6cb;
            color: #721c24;
            padding: 15px;
            margin: 20px;
            border-radius: 6px;
            text-align: center;
        }
        .record-count {
            text-align: center;
            color: #666;
            font-size: 0.9rem;
            margin-bottom: 15px;
        }
    </style>
</head>
<body>
    <div class="container top-title">
        <h1>Module 7 Project Part 2</h1>
```

```
        <h2>Movie Database - <em>READ Operation</em></h2>
        <p class="description">Displaying all movie records from the database.</p>
    </div>

    <div class="container">
<%
        MovieBean movieBean = new MovieBean();
        boolean connected = movieBean.connect();
        int recordCount = 0;

        if (!connected) {
%>
        <div class="error-message">
            <p>Database Connection Error:</p>
            <p><%= movieBean.getErrorMessage() %></p>
        </div>
<%
        } else {
            // Get all movies for display
            boolean success = movieBean.getAllMovies();

            if (!success) {
%>
        <div class="error-message">
            <p>Error retrieving movie data:</p>
            <p><%= movieBean.getErrorMessage() %></p>
        </div>
<%
            } else {
                // First pass: count records
                while (movieBean.next()) {
                    recordCount++;
                }
                movieBean.disconnect();
%>

        <p class="record-count">Total Records: <%= recordCount %></p>

        <table>
            <thead>
                <tr>
```

```
                <th>Movie ID</th>
                <th>Title</th>
                <th>Genre</th>
                <th>Release Year</th>
                <th>Rating</th>
                <th>Director</th>
            </tr>
        </thead>
        <tbody>
<%
            // Second pass: display records
            movieBean.connect();
            movieBean.getAllMovies();
            while (movieBean.next()) {
%>
            <tr>
                <td><%= movieBean.getMovieId() %></td>
                <td><%= movieBean.getTitle() %></td>
                <td><%= movieBean.getGenre() %></td>
                <td><%= movieBean.getReleaseYear() %></td>
                <td><%= movieBean.getRating() %></td>
                <td><%= movieBean.getDirector() %></td>
            </tr>
<%
            }
            movieBean.disconnect();
%>
        </tbody>
    </table>

    <h3>Field Descriptions</h3>
    <ul class="field-descriptions">
        <li><strong>Movie ID:</strong> Unique identifier for each movie (Primary Key, Auto-increment)</li>
        <li><strong>Title:</strong> The name of the movie</li>
        <li><strong>Genre:</strong> The category or type of the movie</li>
        <li><strong>Release Year:</strong> The year the movie was released</li>
        <li><strong>Rating:</strong> MPAA rating (G, PG, PG-13, R, NC-17)</li>
        <li><strong>Director:</strong> The director of the movie</li>
    </ul>
<%
        }
```

```
    }
%>

    <div class="form-actions">
        <a href="movieForm.jsp" class="btn-submit">Add New Movie</a>
        <a href="index.jsp" class="btn-reset">Back to Course Index</a>
    </div>
  </div>

  <footer>
    <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
  </footer>
</body>
</html>
```

deliverables

MODULE 7 PROJECT PART 2

MOVIE DATABASE - READ OPERATION

Displaying all movie records from the database.

Total Records: 10

MOVIE ID	TITLE	GENRE	RELEASE YEAR	RATING	DIRECTOR
1	The Shawshank Redemption	Drama	1994	R	Frank Darabont
2	The Godfather	Crime	1972	R	Francis Ford Coppola
3	The Dark Knight	Action	2008	PG-13	Christopher Nolan
4	Pulp Fiction	Crime	1994	R	Quentin Tarantino
5	The Lord of the Rings: The Return of the King	Fantasy	2003	PG-13	Peter Jackson
6	Forrest Gump	Drama	1994	PG-13	Robert Zemeckis
7	Inception	Sci-Fi	2010	PG-13	Christopher Nolan
8	The Matrix	Sci-Fi	1999	R	The Wachowskis
9	Titanic	Romance	1997	PG-13	James Cameron
10	The Silence of the Lambs	Thriller	1991	R	Jonathan Demme

FIELD DESCRIPTIONS

- **Movie ID:** Unique identifier for each movie (Primary Key, Auto-increment)
- **Title:** The name of the movie
- **Genre:** The category or type of the movie
- **Release Year:** The year the movie was released
- **Rating:** MPAA rating (G, PG, PG-13, R, NC-17)
- **Director:** The director of the movie

Add New Movie | Back to Course Index

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development

MODULE 7 PROJECT PART 2

MOVIE DATABASE - CREATE OPERATION

Add a new movie record to the database.

ADD NEW MOVIE

Movie Title:	Enter movie title
Genre:	-- Select Genre --
Release Year:	e.g., 2024
MPAA Rating:	-- Select Rating --
Director:	Enter director name

Add Movie **Clear Form**

FIELD DESCRIPTIONS

- **Movie Title:** The name of the movie
- **Genre:** The category or type of movie (Action, Comedy, etc.)
- **Release Year:** The year the movie was released
- **MPAA Rating:** Motion picture rating (G, PG, PG-13, R, NC-17)
- **Director:** The director of the movie

View All Movies **Back to Course Index**

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development

MODULE 7 PROJECT PART 2

MOVIE DATABASE - CREATE OPERATION

Add a new movie record to the database.

ADD NEW MOVIE

Movie Title:	Interstellar
Genre:	Sci-Fi
Release Year:	2014
MPAA Rating:	PG-13
Director:	Christopher Nolan

Add Movie | Clear Form

FIELD DESCRIPTIONS

- **Movie Title:** The name of the movie
- **Genre:** The category or type of movie (Action, Comedy, etc.)
- **Release Year:** The year the movie was released
- **MPAA Rating:** Motion picture rating (G, PG, PG-13, R, NC-17)
- **Director:** The director of the movie

View All Movies | Back to Course Index

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development

MODULE 7 PROJECT PART 2

MOVIE DATABASE - CREATE OPERATION

Add a new movie record to the database.

Movie "Interstellar" has been added successfully!

ADD NEW MOVIE

Movie Title:	Enter movie title
Genre:	-- Select Genre --
Release Year:	e.g., 2024
MPAA Rating:	-- Select Rating --
Director:	Enter director name

Add Movie **Clear Form**

FIELD DESCRIPTIONS

- **Movie Title:** The name of the movie
- **Genre:** The category or type of movie (Action, Comedy, etc.)
- **Release Year:** The year the movie was released
- **MPAA Rating:** Motion picture rating (G, PG, PG-13, R, NC-17)
- **Director:** The director of the movie

View All Movies **Back to Course Index**

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development